Task 1: Weighted Average and Pass/Fail

Description: Using args, read three grades (0–100) for a student: quiz, midterm, and final. Compute the weighted average:

- quiz: 20%, midterm: 30%, final: 50%
  Then decide whether the student passes (average ≥ 50) and whether the student qualifies for "Merit" (average ≥ 85).

Input: Three integers: `quiz`, `midterm`, `final` (via `cmd`).

Output:

- Weighted average (as `double`, 2 decimals).
- Status: `PASS` or `FAIL`.
- Merit: `YES` or `NO`.

Task 2: Integer Digit Logic: "Code Validity"

Description: Using args, read a 4-digit positive integer `n`. Determine if it is "valid" using the following rule:

- Let the digits be `a b c d` (thousands to units).
- Valid if `(a + d) == (b + c)` **and** `n` is even.

Input:

One integer `n` (1000–9999).

Output: Print `VALID` or `INVALID`.

Constraints: `n` must be 4 digits (you may assume it is, or you may reject otherwise).

Task 3: Menu-Based Unit Converter (switch-case)

Description: Build a console menu that converts a value based on user choice:

1. Celsius → Fahrenheit
2. Fahrenheit → Celsius
3. Kilometers → Miles
4. Miles → Kilometers

Read the option (1–4) and a numeric value, then output the converted result.

Input:

- `option` (int)
- `value` (double)

Output: Converted value (2 decimals) and a short label.

Constraints: If the option is not 1–4, print `Invalid option`.

Task 4: Change-Making with Greedy Logic (counters)

Description:

Read an amount in cents (0–9999). Compute the minimum number of coins using:

- 200, 100, 50, 20, 10, 5, 2, 1 (cents)

Print how many of each coin are used.

Input:

`amount` (int, in cents)

Constraints:

Use only integer arithmetic.

Output:

Lines like:

```
200c: x
100c: y
...
1c: z
```

Task 5: Sentinel-Controlled Sum (while-loop)

Description: Read integers until the user enters `0` (sentinel). Compute:

- sum of all non-zero numbers
- count of numbers entered (excluding 0)
- average (if count > 0)

Input: A sequence of integers ending with `0`.

Output: Sum, count, and average (2 decimals). If no numbers were entered (first input is 0), print `No data`.

Constraints: Must use a `while` loop.

Task 6: Skip and Stop Rules (for-loop with continue and break)

Description: Read an integer `N` ($N \geq 1$). Then read `N` integers one by one. Apply rules:

- If the value is negative, **skip it** (do not count it, do not add it) using `continue`.
- If the value is `999`, **stop early** using `break`.
  Compute:
- sum of accepted values (non-negative and before 999)
- count of accepted values

Input: `N`, then `N` integers

Output: Accepted count and accepted sum.

Constraints: Must use a `for` loop and both `continue` and `break`.

Task 7: Input Validation Loop (do-while)

Description: Repeatedly prompt the user for a number in the range **[10, 50]**. Keep asking until a valid number is entered. Then print the square of the valid number.

Input: Integers from the user until valid.

Output: The accepted number and its square.

Constraints: Must use a `do-while` loop.

Task 8: Day Type Classifier (switch-case)

Description: Read a day number (1–7). Classify it as:

- `Weekday` for 1–5
- `Weekend` for 6–7
- If outside 1–7, print `Invalid day`.

Input: One integer `day`.

Output: `Weekday`, `Weekend`, or `Invalid day`.

Constraints: Must use `switch-case`. (You may group cases.)

Hints: Use fall-through intentionally: `case 1: case 2: ....`

Task 9: Command-Line Args: Rectangle Report

Description: Use command-line arguments to read two positive numbers: `width` and `height`. Compute:

- area
- perimeter
  Then classify the rectangle:
- `Square` if width == height
- otherwise `Rectangle`

Input: Two command-line arguments: `args[0]`, `args[1]` (numeric).

Output: Width, height, area, perimeter, and classification.

Constraints:

- If `args.length != 2`, print a usage message and exit.
- Convert strings to numbers (`Double.parseDouble` or `Integer.parseInt`).

Task 10: Command-Line Args + Logic: Simple Access Gate

Description: A campus lab has an access rule based on:

- a numeric `id` (integer)
- a `role` string: `"student"`, `"ta"`, or `"staff"`
- a `time` hour (0–23)

All are provided as command-line arguments: `id role time`.

Rules:

- Staff: always allowed.
- TA: allowed if time is between 8 and 20 inclusive.
- Student: allowed only if:
  - time is between 9 and 17 inclusive **and**
  - `id` is even.

  Otherwise, denied.

Input:

Three command-line arguments: `id`, `role`, `time`.

Output: `ACCESS GRANTED` or `ACCESS DENIED` plus a short reason (one phrase).

Constraints:

- If the role is not recognized, deny with `Invalid role`.
- If `args.length != 3`, print usage and exit.